

핸즈온 랩: Apache Spark에서 Delta 테이블 사용하기

Microsoft Fabric Lakehouse의 테이블은 오픈 소스 **Delta Lake** 형식을 기반으로 합니다.

개념 설명: Delta Lake는 데이터 레이크에 저장된 파일(예: Parquet 파일)에 관계형 데이터베이스의 신뢰성과 성능을 더해주는 스토리지 계층입니다. 주요 특징으로는 ACID 트랜잭션(데이터의 원자성, 일관성, 고립성, 지속성 보장), 데이터 버전 관리(Time Travel), 스키마 관리, 배치 및 스트리밍 데이터의 통합 처리 등이 있습니다. 이를 통해 사용자는 데이터 레이크를 마치 데이터 웨어하우스처럼 안정적으로 사용할 수 있습니다.

이 핸즈온 랩에서는 Delta 테이블을 생성하고 SQL 쿼리를 사용하여 데이터를 탐색합니다.

이 실습을 완료하는 데 약 **45분**이 소요됩니다.

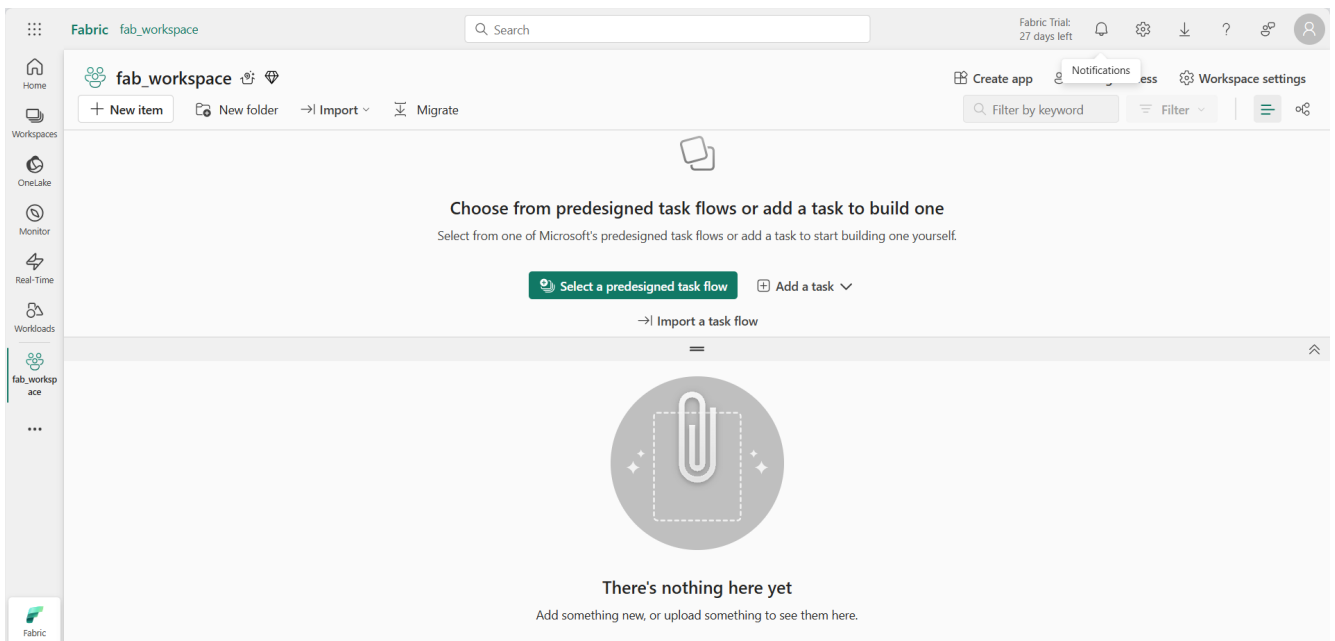
참고

이 실습을 완료하려면 [Microsoft Fabric 테넌트](#)에 대한 액세스 권한이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 용량이 활성화된 테넌트에 Workspace를 만들어야 합니다.

- 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric-developer) `https://app.fabric.microsoft.com/home?experience=fabric-developer` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
- 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
- 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial*, *Premium*, 또는 *Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
- 새 Workspace가 열리면 처음에는 비어 있어야 합니다.



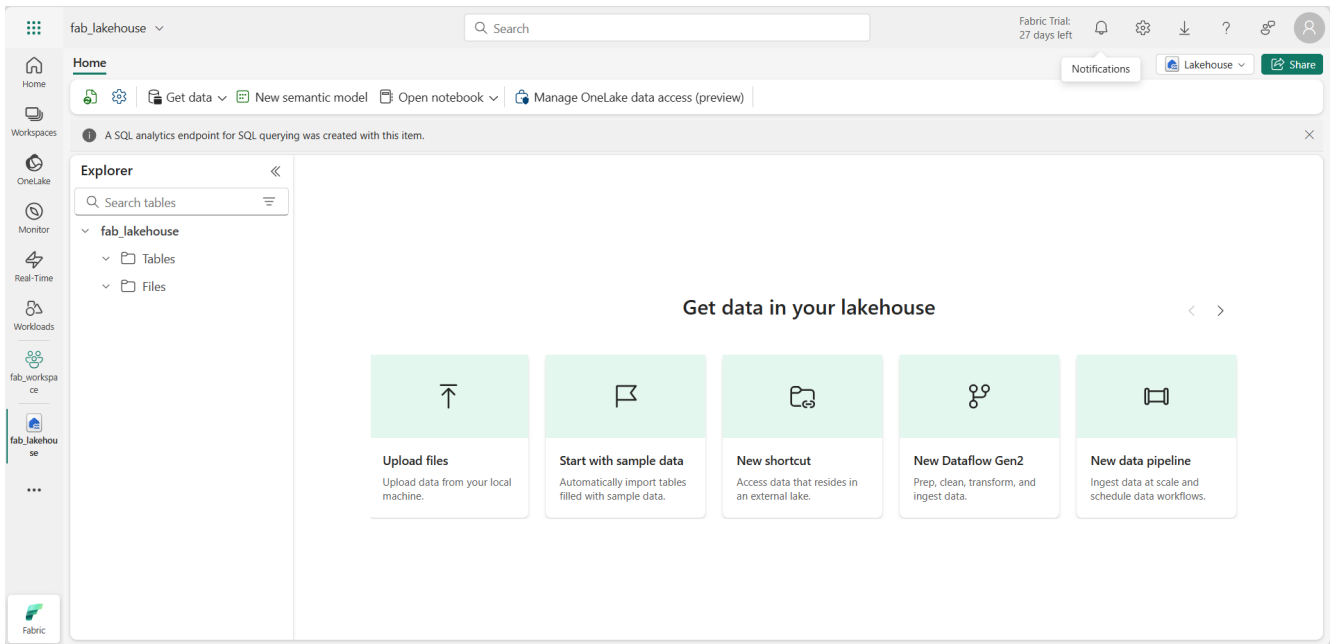
Lakehouse 만들기 및 파일 업로드

이제 Workspace가 준비되었으니 데이터를 위한 데이터 레이크하우스를 만들 차례입니다.

- 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Engineering* 섹션에서 **Lakehouse**를 선택하고, 원하는 고유한 이름을 지정합니다.

참고: **Create** 옵션이 사이드바에 고정되어 있지 않다면, 먼저 줄임표(...) 옵션을 선택해야 합니다.

약 1분 후, 새로운 Lakehouse가 생성됩니다.



2. 새로운 Lakehouse를 살펴보면, 왼쪽의 **Explorer** 창을 통해 Lakehouse의 테이블과 파일을 탐색할 수 있습니다.

이제 Lakehouse로 데이터를 수집할 수 있습니다. 여러 방법이 있지만, 여기서는 텍스트 파일을 로컬 컴퓨터로 다운로드한 후 Lakehouse에 업로드하는 방식을 사용합니다.

1. <https://github.com/MicrosoftLearning/dp-data/raw/main/products.csv>에서 데이터 파일을 다운로드 하여 *products.csv*로 저장합니다.
2. Lakehouse가 열려 있는 웹 브라우저 탭으로 돌아가, **Explorer** 창의 **Files** 폴더 옆에 있는 ... 메뉴를 선택합니다. **New subfolder**를 사용하여 *products*라는 새 하위 폴더를 만듭니다.
3. *products* 폴더의 ... 메뉴에서 로컬 컴퓨터의 *products.csv* 파일을 **upload**합니다.
4. 파일이 업로드된 후, **products** 폴더를 선택하여 아래와 같이 파일이 업로드되었는지 확인합니다.

DataFrame에서 데이터 탐색하기

이제 데이터를 다루기 위해 Fabric Notebook을 만들 수 있습니다.

1. 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Engineering* 섹션에서 **Notebook**을 선택합니다. **Notebook 1**이라는 이름의 새 Notebook이 생성되고 열립니다.
2. Notebook의 이름을 더 설명적인 것으로 변경합니다.
3. 첫 번째 셀을 Markdown 셀로 변환하고 다음 내용을 입력합니다.

```
# Delta Lake tables
Use this notebook to explore Delta Lake functionality
```4. **Explorer** 창에서, 이전에 만든 Lakehouse를 Notebook에 연결합니다.
```

4. 새 코드 셀을 추가하고, 다음 코드를 사용하여 정의된 스키마로 제품 데이터를 DataFrame으로 읽어옵니다.

```

from pyspark.sql.types import StructType, IntegerType, StringType, DoubleType

define the schema
schema = StructType() \
 .add("ProductID", IntegerType(), True) \
 .add("ProductName", StringType(), True) \
 .add("Category", StringType(), True) \
 .add("ListPrice", DoubleType(), True)

df =
spark.read.format("csv").option("header", "true").schema(schema).load("Files/products/
products.csv")
df now is a Spark DataFrame containing CSV data from "Files/products/products.csv".
display(df)

```

#### 코드 설명:

- `from pyspark.sql.types import ...`: 데이터 타입을 정의하기 위해 필요한 클래스들을 가져옵니다.
- `schema = StructType()...`: 데이터프레임의 구조(스키마)를 정의합니다. 각 열의 이름(`ProductID`, `ProductName` 등)과 데이터 타입(`IntegerType`, `StringType` 등), 그리고 Null 허용 여부(`True`)를 지정합니다.
- `spark.read...`: Spark를 사용하여 `products.csv` 파일을 읽습니다.
- `.option("header", "true")`: CSV 파일의 첫 줄을 헤더로 인식합니다.
- `.schema(schema)`: 위에서 정의한 스키마를 적용하여 데이터를 읽도록 지정합니다. 이는 데이터 타입을 정확하게 설정하고 성능을 최적화하는 데 중요합니다.
- `.load(...)`: 지정된 경로의 파일을 로드합니다.
- `display(df)`: 생성된 DataFrame을 테이블 형식으로 출력합니다.

5. 셀 왼쪽의 **Run cell** (▶) 버튼을 사용하여 코드를 실행합니다. 첫 실행 시 Spark 세션이 시작되므로 시간이 다소 걸릴 수 있습니다.
6. 셀 코드 실행이 완료되면, 셀 아래의 출력을 검토합니다.

## Delta 테이블 만들기

DataFrame을 `saveAsTable` 메서드를 사용하여 Delta 테이블로 저장할 수 있습니다. Delta Lake는 **managed** 테이블과 **external** 테이블 생성을 모두 지원합니다.

#### 개념 설명:

- **Managed Table (관리형 테이블)**: Fabric이 스키마 메타데이터와 데이터 파일을 모두 관리하므로 더 높은 성능의 이점을 누릴 수 있습니다. 데이터 파일은 Lakehouse의 `Tables` 폴더 아래에 저장됩니다. 테이블을 삭제하면 메타데이터와 데이터 파일이 모두 삭제됩니다.
- **External Table (외부 테이블)**: 데이터를 외부(예: Lakehouse의 `Files` 폴더)에 저장하고, 메타데이터만 Fabric에서 관리합니다. 테이블을 삭제해도 원본 데이터 파일은 삭제되지 않아 유연성이 높습니다.

## Managed 테이블 만들기

1. 첫 번째 코드 셀의 결과 아래에 새 코드 셀을 추가합니다.
2. Managed Delta 테이블을 만들려면, 새 셀에 다음 코드를 입력하고 실행합니다.

```
df.write.format("delta").saveAsTable("managed_products")
```

#### 코드 설명:

- `df.write`: DataFrame을 저장하기 위한 진입점입니다.
- `.format("delta")`: 데이터를 Delta Lake 형식으로 저장하도록 지정합니다.
- `.saveAsTable("managed_products")`: `managed_products` 라는 이름의 Managed 테이블로 저장합니다. 데이터 파일은 Lakehouse의 `Tables` 디렉터리에 자동으로 생성됩니다.

3. **Explorer** 창에서 `Tables` 폴더를 **Refresh** 하고 `managed_products` 테이블이 생성되었는지 확인합니다.

## External 테이블 만들기

1. **Explorer** 창에서 `Files` 폴더의 ... 메뉴를 선택하고 **Copy ABFS path**를 선택합니다. ABFS 경로는 Lakehouse의 `Files` 폴더에 대한 전체 정규화된 경로입니다.
2. 새 코드 셀에 복사한 ABFS 경로를 붙여넣습니다. 다음 코드의 `abfs_path` 부분에 붙여넣은 경로를 사용하여 코드를 완성합니다.

```
df.write.format("delta").saveAsTable("external_products",
path="abfs_path/external_products")
```

#### 코드 설명:

- `.saveAsTable("external_products", path=...)`: `external_products` 라는 이름의 External 테이블로 저장합니다. `path` 인수는 데이터 파일이 저장될 외부 위치를 명시적으로 지정합니다. 이것이 Managed 테이블과의 핵심적인 차이입니다.

3. 전체 경로는 다음과 유사한 형태가 됩니다.

```
abfss://workspace@tenant-
onelake.dfs.fabric.microsoft.com/lakehouse/Lakehouse/Files/external_products
```

4. 셀을 **Run**하여 DataFrame을 `Files/external_products` 폴더에 External 테이블로 저장합니다.
5. **Explorer** 창에서 `Tables`와 `Files` 폴더를 각각 **Refresh**하여 `external_products` 테이블(메타데이터)과 해당 데이터 파일 폴더가 생성되었는지 확인합니다.

## Managed 테이블과 External 테이블 비교

**핸즈온의 의미:** 이 단계는 Managed 테이블과 External 테이블의 가장 중요한 차이점을 직접 눈으로 확인하는 과정입니다.

`DROP TABLE` 명령을 실행했을 때, Managed 테이블의 데이터는 사라지지만 External 테이블의 데이터는 그대로 남아있는 것을 확인함으로써 두 테이블 타입의 동작 방식을 명확하게 이해할 수 있습니다.

1. 새 코드 셀에서 다음 코드를 실행하여 Managed 테이블의 정보를 확인합니다.

```
%%sql
DESCRIBE FORMATTED managed_products;
```

- 결과에서 `Location` 속성을 확인합니다. OneLake 저장소 위치가 `/Tables/managed_products`로 끝나는 것을 확인하세요.
- 다음과 같이 `DESCRIBE` 명령을 수정하여 External 테이블의 세부 정보를 확인합니다.

```
%%sql
DESCRIBE FORMATTED external_products;
```

- 결과에서 `Location` 속성을 확인합니다. OneLake 저장소 위치가 `/Files/external_products`로 끝나는 것을 확인하세요.
- 새 코드 셀에서 다음 코드를 실행하여 두 테이블을 모두 삭제합니다.

```
%%sql
DROP TABLE managed_products;
DROP TABLE external_products;
```

- Explorer** 창에서 `Tables` 폴더를 **Refresh**하여 두 테이블이 모두 사라졌는지 확인합니다.
- Explorer** 창에서 `Files` 폴더를 **Refresh**하고 `external_products` 파일 폴더가 **삭제되지 않았는지** 확인합니다.

External 테이블의 메타데이터는 삭제되었지만, 데이터 파일은 그대로 남아있습니다.

## SQL을 사용하여 Delta 테이블 만들기

이제 `%%sql` 매직 커맨드를 사용하여 SQL로 직접 Delta 테이블을 생성해 보겠습니다.

- 새 코드 셀을 추가하고 다음 코드를 실행합니다.

```
%%sql
CREATE TABLE products
USING DELTA
LOCATION 'Files/external_products';
```

### 코드 설명:

- `CREATE TABLE products`: `products` 라는 새 테이블을 생성합니다.
  - `USING DELTA`: 이 테이블이 Delta Lake 형식을 사용하도록 지정합니다.
  - `LOCATION 'Files/external_products'`: 기존에 존재하는 `external_products` 폴더의 Delta 파일들을 사용하여 테이블을 생성합니다. 이는 삭제되었던 External 테이블의 메타데이터를 다시 만드는 것과 같습니다.
- `Tables` 폴더를 **Refresh**하여 `products` 라는 새 테이블이 생성되었는지 확인합니다.
  - 새 코드 셀에서 다음 코드를 실행하여 데이터를 확인합니다.

```
%%sql
SELECT * FROM products;
```

## 테이블 버전 관리 탐색 (Time Travel)

**개념 설명: Time Travel**은 Delta Lake의 가장 강력한 기능 중 하나입니다. `_delta_log` 폴더에 저장된 트랜잭션 기록을 사용하여, 특정 시점이나 특정 버전의 테이블 상태로 데이터를 조회할 수 있습니다. 이를 통해 실수로 데이터를 잘못 변경했거나 과거 데이터를 분석해야 할 때 매우 유용합니다.

**핸즈온의 의미:** 이 섹션에서는 데이터를 직접 업데이트하고, 트랜잭션 기록을 조회한 뒤, 실제로 과거 버전의 데이터를 불러오는 과정을 체험합니다. 이를 통해 Delta Lake의 Time Travel 기능이 어떻게 동작하는지 직관적으로 이해할 수 있습니다.

1. 새 코드 셀을 추가하고, 산악 자전거(`Mountain Bikes`)의 가격을 10% 할인하는 다음 코드를 실행합니다.

```
%%sql
UPDATE products
SET ListPrice = ListPrice * 0.9
WHERE Category = 'Mountain Bikes';
```

2. 새 코드 셀에서 다음 코드를 실행하여 테이블의 트랜잭션 기록을 확인합니다.

```
%%sql
DESCRIBE HISTORY products;
```

결과는 테이블에 기록된 트랜잭션의 내역을 보여줍니다. `version` 0은 테이블 생성, `version` 1은 `UPDATE` 작업에 해당합니다.

3. 새 코드 셀에서 다음 코드를 실행하여 현재 데이터와 초기 버전(version 0) 데이터를 비교합니다.

```
delta_table_path = 'Files/external_products'
Get the current data
current_data = spark.read.format("delta").load(delta_table_path)
display(current_data)

Get the version 0 data
original_data = spark.read.format("delta").option("versionAsOf", 0).load(delta_table_path)
display(original_data)
```

#### 코드 설명:

- `.option("versionAsOf", 0)`: Delta 테이블을 읽을 때, `version` 0 시점의 데이터를 가져오도록 지정하는 옵션입니다.

두 개의 결과 세트가 반환됩니다. 하나는 가격 인하 후의 데이터, 다른 하나는 원본 데이터를 보여줍니다.

## SQL 쿼리로 Delta 테이블 데이터 분석하기

1. 새 코드 셀을 추가하고, 다음 코드를 실행하여 `products` 테이블로부터 임시 뷰(`temporary view`)를 생성하고 표시합니다.

```
%%sql
-- Create a temporary view
CREATE OR REPLACE TEMPORARY VIEW products_view
AS
 SELECT Category, COUNT(*) AS NumProducts, MIN(ListPrice) AS MinPrice,
 MAX(ListPrice) AS MaxPrice, AVG(ListPrice) AS AvgPrice
 FROM products
 GROUP BY Category;

SELECT *
FROM products_view
ORDER BY Category;
```

2. 새 코드 셀을 추가하고, 다음 코드를 실행하여 제품 수 기준으로 상위 10개 카테고리를 반환합니다.

```
%%sql
SELECT Category, NumProducts
FROM products_view
ORDER BY NumProducts DESC
LIMIT 10;
```

3. 데이터가 반환되면 + **New chart**를 선택하여 제안된 차트 중 하나를 표시합니다.

또는 PySpark를 사용하여 SQL 쿼리를 실행할 수도 있습니다.

1. 새 코드 셀을 추가하고 다음 코드를 실행합니다.

```
from pyspark.sql.functions import col, desc

df_products = spark.sql("SELECT Category, MinPrice, MaxPrice, AvgPrice FROM
products_view").orderBy(col("AvgPrice").desc())
display(df_products.limit(6))
```

## 스트리밍 데이터에 Delta 테이블 사용하기

**개념 설명:** Delta Lake는 스트리밍 데이터를 완벽하게 지원합니다. Delta 테이블은 Spark 구조적 스트리밍 API를 사용하여 생성된 데이터 스트림의 **sink**(데이터가 최종적으로 저장되는 목적지) 또는 **source**(데이터를 읽어오는 원천)가 될 수 있습니다. 이는 실시간으로 들어오는 데이터를 안정적으로 처리하고 저장하는 데 매우 중요합니다.

1. 새 코드 셀을 추가하고 다음 코드를 실행하여 스트리밍 소스를 설정합니다.

```
from notebookutils import mssparkutils
from pyspark.sql.types import *
from pyspark.sql.functions import *

Create a folder
inputPath = 'Files/data/'
mssparkutils.fs.mkdirs(inputPath)
```

```
Create a stream that reads data from the folder, using a JSON schema
jsonSchema = StructType([
 StructField("device", StringType(), False),
 StructField("status", StringType(), False)
])
iotstream = spark.readStream.schema(jsonSchema).option("maxFilesPerTrigger",
1).json(inputPath)

Write some event data to the folder
device_data = '''{"device":"Dev1","status":"ok"}
{"device":"Dev1","status":"ok"}
{"device":"Dev1","status":"ok"}
{"device":"Dev2","status":"error"}
{"device":"Dev1","status":"ok"}
{"device":"Dev1","status":"error"}
{"device":"Dev2","status":"ok"}
{"device":"Dev2","status":"error"}
{"device":"Dev1","status":"ok"}'''

mssparkutils.fs.put(inputPath + "data.txt", device_data, True)

print("Source stream created...")
```

2. 새 코드 셀에서 다음 코드를 추가하고 실행하여 스트림 데이터를 Delta 테이블로 보냅니다.

```
Write the stream to a delta table
delta_stream_table_path = 'Tables/iotdevicedata'
checkpointpath = 'Files/delta/checkpoint'
deltastream = iotstream.writeStream.format("delta").option("checkpointLocation",
checkpointpath).start(delta_stream_table_path)
print("Streaming to delta sink...")
```

#### 코드 설명:

- `iotstream.writeStream`: 읽기 스트림(`iotstream`)을 쓰기 스트림으로 전환합니다.
- `.format("delta")`: 스트림의 출력 형식을 Delta로 지정합니다.
- `.option("checkpointLocation", ...)`: 스트림 처리 상태를 저장할 체크포인트 위치를 지정합니다. 이는 스트림 장애 발생 시 중단된 지점부터 안정적으로 재시작하기 위해 필수적입니다.
- `.start(delta_stream_table_path)`: 스트림 쓰기를 `Tables/iotdevicedata` 경로로 시작합니다. `Tables` 경로에 쓰기 때문에 `IotDeviceData` 라는 이름의 테이블이 자동으로 생성됩니다.

3. 새 코드 셀에서 다음 코드를 실행하여 `IotDeviceData` 테이블을 쿼리합니다.

```
%%sql
SELECT * FROM IotDeviceData;
```

4. 새 코드 셀에서 다음 코드를 실행하여 스트리밍 소스에 더 많은 데이터를 추가합니다.



```
Add more data to the source stream
more_data = '''{"device":"Dev1","status":"ok"}
{"device":"Dev1","status":"ok"}
...'''
mssparkutils.fs.put(inputPath + "more-data.txt", more_data, True)
```

5. 이전 `SELECT * FROM IotDeviceData;` 쿼리 셀을 다시 실행합니다. 이제 테이블에 추가된 데이터가 포함되어 있는 것을 확인할 수 있습니다.
6. 새 코드 셀에서 스트림을 중지하는 코드를 추가하고 셀을 실행합니다.

```
deltastream.stop()
```

## 리소스 정리

이 실습에서는 Microsoft Fabric에서 Delta 테이블을 사용하는 방법을 배웠습니다. Lakehouse 탐색을 마쳤다면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

1. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
2. 도구 모음의 ... 메뉴에서 **Workspace settings**를 선택합니다.
3. General 섹션에서 **Remove this workspace**를 선택합니다.